

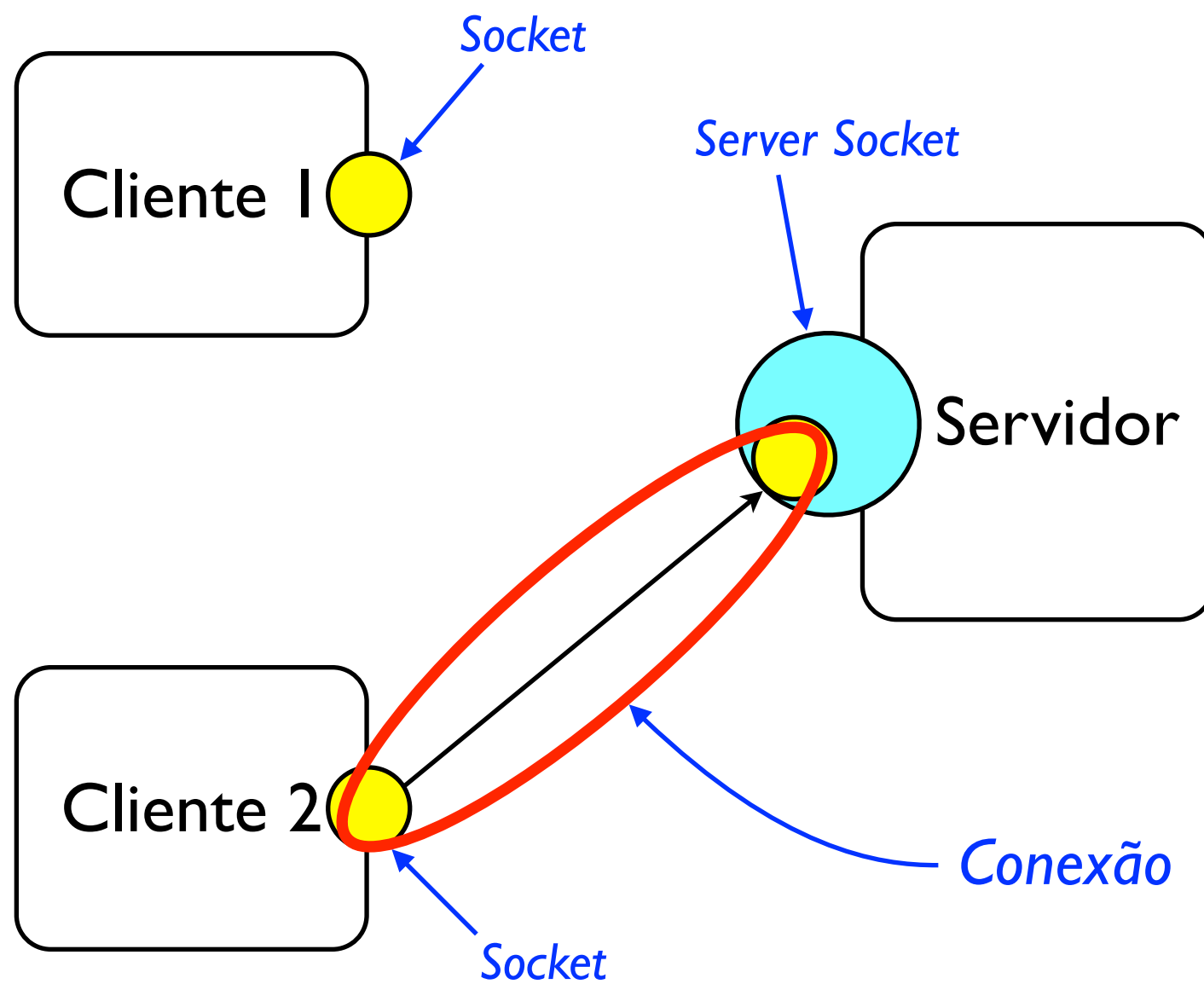
Cliente-Servidor com Sockets TCP

Sistemas Distribuídos

Paradigma cliente - servidor

- Servidor oferece um serviço ao cliente (e.g. servidor web envia html da página pedida, servidor de mail entrega novos emails, ...)
- Cliente efetua pedidos ao servidor e trata as respostas (e.g. cliente do browser mostra a página web recebida do servidor, ...)
 - O cliente tipicamente inicia o contacto com o servidor
- Comunicação deve ser fiável (sem perda de dados e com entrega ordenada de mensagens) → canais TCP

Paradigma cliente - servidor



- Servidor fica à espera de ligações num determinado porto;
- Quando o cliente se liga ao servidor é estabelecida uma nova conexão bidireccional;
- Socket representa um extremo de uma conexão.
- Numa ligação, existem dois extremos (socket)

- Classes e métodos relevantes em JAVA:
- (servidor) `java.net.ServerSocket`:
 - construtor: `ServerSocket(int port)`
 - outros métodos: `accept()`, `close()`
- (cliente) `java.net.Socket`:
 - construtor: `Socket(String host, int port)`
 - outros métodos: `connect()`, `getInputStream()`, `getOutputStream()`
- ler / escrever no socket: `BufferedReader`, `InputStreamReader` / `PrintWriter`

Cliente

Esqueleto:

JAVA:

criar socket
e ligação com
o servidor

```
Socket socket = new Socket(address, port);
```

abrir canais
de escrita e
leitura no
socket

```
BufferedReader in = new BufferedReader(new  
InputStreamReader(socket.getInputStream()));
```

```
PrintWriter out = new PrintWriter(  
socket.getOutputStream());
```

ler e escrever
nos canais de
acordo com o
protocolo da
aplicação

```
while(...) {  
    out.println(...);  
    out.flush();  
    ... = in.readLine();  
}
```

fechar socket
e respectivos
canais

```
socket.shutdownOutput();  
socket.shutdownInput();  
socket.close();
```

Servidor

Esqueleto:

JAVA:

criar novo
server socket
num dado porto

```
ServerSocket sSock = new ServerSocket(port);
```

aceitar conexões
indefinidamente

```
while (true)  
{
```

bloquear até que
uma conexão
seja estabelecida

```
Socket clSock = sSock.accept();
```

abrir canais
de escrita e
leitura no socket

```
BufferedReader in = new BufferedReader(new  
InputStreamReader(clSock.getInputStream()));
```

```
PrintWriter out = new PrintWriter(  
clSock.getOutputStream());
```

ler e escrever nos
canais de acordo
com o protocolo
da aplicação

```
while(...) {  
    ... = in.readLine();  
    out.println(...);  
    out.flush();  
}
```

fechar socket
e respectivos
canais

```
clSock.shutdownOutput();  
clSock.shutdownInput();  
clSock.close();
```

```
}
```